

Introducción a Git (local): control de versiones desde cero

1. Introducción a Git (local): control de versiones desde cero

Qué es Git, y por qué existe

Git es un sistema de control de versiones

Lleva el historial de un proyecto (qué cambios se han hecho), sabe cuándo y quién los hizo, y permite volver atrás si algo se rompe. Como una máquina del tiempo para tu proyecto.

Sin Git, cuando programamos suele pasar esto:

«¿Qué he tocado?»

Antes funcionaba, y ya no sabes qué cambiaste.

final.zip, final_final.zip...

Copias «por si acaso» en vez de un historial real.

Cambios mezclados

Ya no sabes separar lo que hiciste de cada cosa.

Git resuelve todo eso porque crea un historial ordenado y recuperable.

Qué es un commit, y por qué es tan importante

Un commit es un punto de guardado del proyecto

Como en un videojuego: estás jugando (programando), llegas a un punto estable, guardas la partida (haces un commit), y si luego la lías, vuelves a esa partida.

¿Qué lleva un commit?

Una «foto»

del estado de los archivos

El autor

tu nombre y tu email

La fecha

cuándo se hizo

Un mensaje

qué y para qué

Un buen mensaje dice qué has hecho y para qué: «Corrige cálculo del total con descuento», no «cambios».

La idea que más cuesta: staging (la bandeja)

Git no funciona como el «guardar» de Word. El staging area (o index) es la lista de cambios seleccionados que van a entrar en el próximo commit: lo que no está en staging, todavía no se guarda.



Escritorio: donde editas y rompes cosas. Bandeja: lo que ya está listo para «registrarse». Álbum: donde quedan guardadas las versiones (los commits).

Por qué existe la bandeja

Porque en la vida real los cambios se mezclan: arreglas un bug, cambias un texto del README, retocas un CSS... Si haces un commit con todo junto, el historial se vuelve difícil de usar.

Sin staging: todo junto

```
git add .  
git commit -m "Cambios"
```

Un «batiburrillo»

Mezcla el bug, el README y el CSS en un solo commit. Cuesta entenderlo o deshacerlo por partes.

Con staging: uno por idea

```
git add Pedido.java  
git commit -m "Corrige bug en pedido"  
  
git add README.md  
git commit -m "Actualiza instrucciones"
```

Historial claro

Cada commit se entiende de un vistazo y se puede deshacer por partes.

Estados de un archivo (lo que cuenta git status)

Untracked

Git ve el archivo, pero aún no lo sigue: es nuevo.

Modified

Lo has cambiado desde el último commit.

Staged

Lo has puesto en la bandeja para el próximo commit.

Committed

Ya está guardado en el historial.

Untracked /
Modified

git add →

Staged

git commit →

Committed

Creas hola.txt → untracked. git add hola.txt → staged. git commit → committed. Editas hola.txt → modified otra vez.

Instalación y configuración básica

```
git --version
```

Windows

Git for Windows, incluye Git Bash.

macOS

git --version te ofrece instalar
Command Line Tools.

Linux

sudo apt install git

Configuración básica (una sola vez)

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tu@email.com"  
git config --global init.defaultBranch main
```

Evita OneDrive / Google Drive / iCloud

Crear repositorios dentro de carpetas sincronizadas en la nube puede provocar cambios «fantasma» al cambiar de versión.

Crear un repositorio: git init

Entras en la carpeta del proyecto e inicializas Git

```
cd ~/Documents/pruebas_git  
git init
```

On branch main

Estás en la rama main.

No commits yet

Todavía no has hecho ningún commit.

La carpeta .git

Es la «memoria» del repositorio: historial, ramas y metadatos.

No borres ni edites .git a mano

Si eliminas .git, el proyecto pierde el historial aunque los archivos sigan ahí.

Comandos esenciales: status y diff

git status — «¿qué está pasando?»

```
git status
```

Dos bloques importantes

Changes to be committed → está en staging. Untracked files → Git lo ve, pero aún no lo sigues (no has hecho git add).

git diff — «¿qué ha cambiado exactamente?»

```
git diff
```

Línea a línea

Muestra los cambios respecto al último commit. Las líneas con + son lo nuevo que has añadido.

Regla de oro: antes de git add o git commit, mira git status para confirmar qué vas a guardar.

Comandos esenciales: add y commit

git add — «pongo esto en la bandeja»

```
git add adios.txt hola.txt
```

Prepara, no guarda

git add no guarda nada todavía: solo prepara archivos para el próximo commit.

git commit — «guardo la partida»

```
git commit -m "Añado hola y adios"
```

Solo lo de staging

Un commit guarda solo lo que hayas puesto en staging con git add — nada más.

git add . mete en staging TODO lo nuevo o modificado

Al principio es mejor añadir archivo a archivo, para no llevarte cosas sin querer.

git log — «muéstrame el historial»

git log

```
git log
```

HEAD -> main

Los commits salen en orden, el más reciente arriba.
HEAD -> main indica en qué rama estás y a qué commit apunta.

git log --oneline — una línea por commit

```
git log --oneline
```

El hash abreviado

Los 7 caracteres del inicio (1cd8f5a...) identifican cada commit. Los usarás en git show y git revert.

Cada commit tiene un hash único, una cadena larga de letras y números — con los 7 primeros caracteres es suficiente.

«Me he equivocado»: descartar cambios con git restore

A) Descartar cambios de un archivo

```
git restore hola.txt
```

«Deshaz cambios del archivo»

Vuelve al estado del último commit. No toca el staging.

B) Sacar un archivo del staging

```
git restore --staged hasta-luego.txt
```

«Saca de la bandeja»

Lo saca del staging sin borrar sus cambios. No toca el directorio de trabajo.

git restore borra cambios no guardados en commits

Si esos cambios te importan, guarda antes (commit) o copia el archivo.

Truco mental: `restore --staged` = «saca de la bandeja»; `restore` (sin `--staged`) = «deshaz cambios del archivo».

Volver a una versión anterior: ¿qué necesitas?

Antes de actuar, la pregunta clave es: ¿qué quiero exactamente? Antes de nada, mira el historial con `git log --oneline` para saber qué hashes tienes.

Ver qué cambió un commit

`git show <hash>`

Recuperar un archivo concreto

`git restore --source=<hash>
<archivo>`

Deshacer un commit entero

¿Ya lo has subido a GitHub?

Sí, está compartido → `git revert`

Seguro: no borra historial, añade un commit que deshace.

No, es solo local → `git reset`

Potente: puede borrar commits, como si nunca hubieran existido.

Deshacer un commit entero: git revert

```
git revert 1cd8f5a
```

Crea un commit nuevo que deshace el anterior

Es la opción más segura: no borra nada del historial, solo añade un paso nuevo que revierte lo que se hizo. Git genera el mensaje automáticamente.

```
commit "Primer commit"  
commit "Añade el cuarto llibre"  
commit "Elimina un llibre"  
commit "Revert: Elimina un llibre" ← nuevo
```

Cuándo usarlo: siempre que hayas subido el repo a GitHub o trabajado con otras personas. Al no reescribir el historial, no crea conflictos con el trabajo de los demás.

Mover el repo a un commit anterior: git reset

git reset mueve HEAD hacia atrás y borra commits — como si nunca hubieran existido. Solo recomendado si el repo es estrictamente local. La diferencia entre variantes: ¿adónde van los cambios del commit eliminado?

Variante	¿Adónde van los cambios?	git status muestra	¿Recuperable?
git reset --soft HEAD~1	Staging	M archivo (zona staging)	Sí
git reset HEAD~1 (--mixed)	Escritorio de trabajo	M archivo (zona trabajo)	Sí
git reset --hard HEAD~1	Se borran	(nada)	No

--hard: sin vuelta atrás

Los cambios del commit eliminado se pierden para siempre. No hay papelera ni deshacer.

.gitignore: lo que no queremos guardar

Cosas que NO queremos guardar en el historial

```
# Archivos de log
*.log

# Carpeta de compilación de Java
target/

# Variables de entorno y credenciales – NUNCA deben subirse
.env
```

Credenciales, nunca al historial

Contraseñas, tokens y claves API no deben acabar en un .gitignore mal puesto: si ya se han subido, cambiar el .gitignore no las borra del historial.

Ideas clave

1

Git es un sistema de control de versiones: historial, quién y cuándo, y poder volver atrás.

2

Un commit es un punto de guardado, con autor, fecha y mensaje.

3

El staging es la lista de cambios seleccionados para el próximo commit: solo se guarda lo que está ahí.

4

Estados de un archivo: untracked → modified → staged (git add) → committed (git commit).

5

git status, git diff, git add, git commit y git log son los comandos del día a día.

6

restore (sin --staged) deshace cambios del archivo; restore --staged solo lo saca de la bandeja.

7

revert crea un commit nuevo y es seguro para historial compartido; reset mueve HEAD y solo es seguro en local.

8

reset --soft deja los cambios en staging; --mixed en el escritorio; --hard los borra para siempre.